

Competitive Programming

Section 73 Editorial: Convex Geometry And Minkowski Sums

Competitive Programming Course

Table of contents

1	How to use this editorial	3
2	A. Hull Statistics	4
2.1	How to find it	4
2.2	Derivation and correctness	4
2.3	Failure-focused tests	4
2.4	C++	4
2.5	Python	5
3	B. Farthest Pair	7
3.1	How to find it	7
3.2	Derivation and correctness	7
3.3	Failure-focused tests	7
3.4	C++	7
3.5	Python	9
4	C. Sum Polygon Queries	11
4.1	How to find it	11
4.2	Derivation and correctness	11
4.3	Failure-focused tests	11
4.4	C++	11
4.5	Python	14
5	D. Convex Polygon Distance	17
5.1	Restatement and decisive reduction	17
5.2	Algorithm	17
5.3	Correctness proof	17
5.4	Complexity, mistakes, and tests	18
5.5	C++	18
5.6	Python	20
6	Stress testing and carry-forward	22

1 How to use this editorial

Recover the geometric invariant and degeneracy policy before comparing code.

2 A. Hull Statistics

2.1 How to find it

The raw set has no useful cyclic order. Ask which points can maximize a dot product in some direction; only extreme points survive, suggesting a convex hull before any area computation.

2.2 Derivation and correctness

Sort and deduplicate. The lower and upper stacks retain only strict left turns. Every popped point lies on or inside the segment joining its neighbors, so it cannot be an extreme vertex. The two chains contain every extreme point once. Shoelace on that canonical cycle gives doubled area. Time is $O(n \log n)$.

2.3 Failure-focused tests

Compare with the independent tiny brute-force oracle. Include collinearity, duplicates, equal event coordinates, boundary contact, and the smallest legal instance.

2.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
struct P {
    int64 x, y;
};
bool operator<(P a, P b) { return tie(a.x, a.y) < tie(b.x, b.y); }
bool operator==(P a, P b) { return a.x == b.x && a.y == b.y; }
P operator+(P a, P b) { return {a.x + b.x, a.y + b.y}; }
P operator-(P a, P b) { return {a.x - b.x, a.y - b.y}; }
int64 cross(P a, P b) { return a.x * b.y - a.y * b.x; }
int64 cross(P a, P b, P c) { return cross(b - a, c - a); }
int64 dist2(P a, P b) {
    int64 x = a.x - b.x, y = a.y - b.y;
    return x * x + y * y;
}
vector<P> hull(vector<P> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
```

```

if (p.size() <= 1)
    return p;
vector<P> h;
for (P x : p) {
    while (h.size() >= 2 && cross(h[h.size() - 2], h.back(), x) <= 0)
        h.pop_back();
    h.push_back(x);
}
size_t lower = h.size();
for (int i = (int)p.size() - 2; i >= 0; --i) {
    P x = p[i];
    while (h.size() > lower && cross(h[h.size() - 2], h.back(), x) <= 0)
        h.pop_back();
    h.push_back(x);
}
h.pop_back();
return h;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n))
        return 0;
    vector<P> p(n);
    for (auto &q : p)
        cin >> q.x >> q.y;
    auto h = hull(p);
    long long a = 0;
    for (int i = 0; i < (int)h.size(); ++i)
        a += cross(h[i], h[(i + 1) % h.size()]);
    cout << h.size() << ' ' << llabs(a) << '\n';
}

```

2.5 Python

```

import sys

def cross(a, b, c=None):
    if c is None:
        return a[0] * b[1] - a[1] * b[0]
    return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])

def hull(points):
    points = sorted(set(points))

```

```

if len(points) <= 1:
    return points
lower = []
for point in points:
    while len(lower) >= 2 and cross(lower[-2], lower[-1], point) <= 0:
        lower.pop()
    lower.append(point)
upper = []
for point in reversed(points):
    while len(upper) >= 2 and cross(upper[-2], upper[-1], point) <= 0:
        upper.pop()
    upper.append(point)
return lower[:-1] + upper[:-1]

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    h = hull(list(zip(data[1::2], data[2::2])))
    area = (
        abs(sum(cross(h[i], h[(i + 1) % len(h)]) for i in range(len(h))))
        if h
        else 0
    )
    print(len(h), area)

if __name__ == "__main__":
    main()

```

3 B. Farthest Pair

3.1 How to find it

The quadratic search remains quadratic even after building a hull. Plot the area against one fixed hull edge: convexity makes the farthest supporting vertex move in one direction, which is the signal for calipers.

3.2 Derivation and correctness

Only hull vertices can form a farthest pair. For each directed hull edge, advance the opposite pointer while its cross product with that edge increases. Convexity makes this value unimodal and makes the pointer monotone. Rotating calipers visits every antipodal candidate in $O(h)$ after the hull.

3.3 Failure-focused tests

Compare with the independent tiny brute-force oracle. Include collinearity, duplicates, equal event coordinates, boundary contact, and the smallest legal instance.

3.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
struct P {
    int64 x, y;
};
bool operator<(P a, P b) { return tie(a.x, a.y) < tie(b.x, b.y); }
bool operator==(P a, P b) { return a.x == b.x && a.y == b.y; }
P operator+(P a, P b) { return {a.x + b.x, a.y + b.y}; }
P operator-(P a, P b) { return {a.x - b.x, a.y - b.y}; }
int64 cross(P a, P b) { return a.x * b.y - a.y * b.x; }
int64 cross(P a, P b, P c) { return cross(b - a, c - a); }
int64 dist2(P a, P b) {
    int64 x = a.x - b.x, y = a.y - b.y;
    return x * x + y * y;
}
vector<P> hull(vector<P> p) {
    sort(p.begin(), p.end());
```

```

p.erase(unique(p.begin(), p.end()), p.end());
if (p.size() <= 1)
    return p;
vector<P> h;
for (P x : p) {
    while (h.size() >= 2 && cross(h[h.size() - 2], h.back(), x) <= 0)
        h.pop_back();
    h.push_back(x);
}
size_t lower = h.size();
for (int i = (int)p.size() - 2; i >= 0; --i) {
    P x = p[i];
    while (h.size() > lower && cross(h[h.size() - 2], h.back(), x) <= 0)
        h.pop_back();
    h.push_back(x);
}
h.pop_back();
return h;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n))
        return 0;
    vector<P> p(n);
    for (auto &q : p)
        cin >> q.x >> q.y;
    auto h = hull(p);
    int m = h.size();
    if (m < 2) {
        cout << 0 << '\n';
        return 0;
    }
    long long ans = 0;
    int j = 1;
    for (int i = 0; i < m; ++i) {
        int ni = (i + 1) % m;
        while (cross(h[ni] - h[i], h[(j + 1) % m] - h[i]) >
                cross(h[ni] - h[i], h[j] - h[i]))
            j = (j + 1) % m;
        ans = max({ans, dist2(h[i], h[j]), dist2(h[ni], h[j])});
    }
    cout << ans << '\n';
}

```

3.5 Python

```
import sys

def cross(a, b, c=None):
    if c is None:
        return a[0] * b[1] - a[1] * b[0]
    return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])

def hull(points):
    points = sorted(set(points))
    if len(points) <= 1:
        return points
    lower = []
    for point in points:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], point) <= 0:
            lower.pop()
        lower.append(point)
    upper = []
    for point in reversed(points):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], point) <= 0:
            upper.pop()
        upper.append(point)
    return lower[:-1] + upper[:-1]

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    h = hull(list(zip(data[1::2], data[2::2])))
    m = len(h)
    if m < 2:
        print(0)
        return

    def sub(a, b):
        return (a[0] - b[0], a[1] - b[1])

    def d2(a, b):
        return (a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2

    ans = 0
    j = 1
    for i in range(m):
        ni = (i + 1) % m
        e = sub(h[ni], h[i])
        while cross(e, sub(h[(j + 1) % m], h[i])) > cross(e, sub(h[j], h[i])):
```

```
        j = (j + 1) % m
        ans = max(ans, d2(h[i], h[j]), d2(h[ni], h[j]))
    print(ans)

if __name__ == "__main__":
    main()
```

4 C. Sum Polygon Queries

4.1 How to find it

The quantifier “choose one point from each polygon and add them” names a Minkowski sum. Looking at one support direction shows both why the sum is convex and why its boundary edges can be merged by angle.

4.2 Derivation and correctness

Support functions add under Minkowski sum, proving convexity and the edge-angle merge. Starting at the sum of lowest vertices and merging the sorted edge vectors traces the entire boundary once. A fan binary search then locates each query in $O(\log(n+m))$.

4.3 Failure-focused tests

Compare with the independent tiny brute-force oracle. Include collinearity, duplicates, equal event coordinates, boundary contact, and the smallest legal instance.

4.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
struct P {
    int64 x, y;
};
bool operator<(P a, P b) { return tie(a.x, a.y) < tie(b.x, b.y); }
bool operator==(P a, P b) { return a.x == b.x && a.y == b.y; }
P operator+(P a, P b) { return {a.x + b.x, a.y + b.y}; }
P operator-(P a, P b) { return {a.x - b.x, a.y - b.y}; }
int64 cross(P a, P b) { return a.x * b.y - a.y * b.x; }
int64 cross(P a, P b, P c) { return cross(b - a, c - a); }
int64 dist2(P a, P b) {
    int64 x = a.x - b.x, y = a.y - b.y;
    return x * x + y * y;
}
vector<P> hull(vector<P> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
```

```

    if (p.size() <= 1)
        return p;
    vector<P> h;
    for (P x : p) {
        while (h.size() >= 2 && cross(h[h.size() - 2], h.back(), x) <= 0)
            h.pop_back();
        h.push_back(x);
    }
    size_t lower = h.size();
    for (int i = (int)p.size() - 2; i >= 0; --i) {
        P x = p[i];
        while (h.size() > lower && cross(h[h.size() - 2], h.back(), x) <= 0)
            h.pop_back();
        h.push_back(x);
    }
    h.pop_back();
    return h;
}

vector<P> normalize(vector<P> p) {
    if (p.size() <= 1)
        return p;
    if (cross(p[0], p[1], p[2]) < 0)
        reverse(p.begin(), p.end());
    int at = min_element(p.begin(), p.end(),
        [](P a, P b) { return tie(a.y, a.x) < tie(b.y, b.x); }) -
        p.begin();
    rotate(p.begin(), p.begin() + at, p.end());
    return p;
}

vector<P> minkowski(vector<P> a, vector<P> b) {
    a = normalize(a);
    b = normalize(b);
    if (a.size() < 3 || b.size() < 3) {
        vector<P> s;
        for (P x : a)
            for (P y : b)
                s.push_back(x + y);
        return hull(s);
    }
    int n = a.size(), m = b.size(), i = 0, j = 0;
    vector<P> r = {a[0] + b[0]};
    while (i < n || j < m) {
        P ea = i < n ? a[(i + 1) % n] - a[i] : P{0, 0};
        P eb = j < m ? b[(j + 1) % m] - b[j] : P{0, 0};
        long long z = (i < n && j < m) ? cross(ea, eb) : 0;
        P step;
        if (j == m || (i < n && z > 0))
            step = ea, ++i;
        else if (i == n || z < 0)
            step = eb, ++j;
    }
}

```

```

        else
            step = ea + eb, ++i, ++j;
            r.push_back(r.back() + step);
    }
    r.pop_back();
    return hull(r);
}

int locate(const vector<P> &p, P q) {
    int n = p.size();
    if (n == 1)
        return q == p[0] ? 0 : -1;
    if (n == 2)
        return cross(p[0], p[1], q) == 0 && min(p[0].x, p[1].x) <= q.x &&
            q.x <= max(p[0].x, p[1].x) && min(p[0].y, p[1].y) <= q.y &&
            q.y <= max(p[0].y, p[1].y)
            ? 0
            : -1;
    long long a = cross(p[0], p[1], q), b = cross(p[0], p.back(), q);
    if (a < 0 || b > 0)
        return -1;
    if (a == 0)
        return dist2(p[0], q) <= dist2(p[0], p[1]) ? 0 : -1;
    if (b == 0)
        return dist2(p[0], q) <= dist2(p[0], p.back()) ? 0 : -1;
    int l = 1, r = n - 1;
    while (r - l > 1) {
        int mid = (l + r) / 2;
        if (cross(p[0], p[mid], q) >= 0)
            l = mid;
        else
            r = mid;
    }
    long long z = cross(p[l], p[(l + 1) % n], q);
    return z < 0 ? -1 : (z == 0 ? 0 : 1);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    if (!(cin >> n >> m >> q))
        return 0;
    vector<P> a(n), b(m);
    for (auto &x : a)
        cin >> x.x >> x.y;
    for (auto &x : b)
        cin >> x.x >> x.y;
    auto s = minkowski(a, b);
    while (q--) {
        P x;
        cin >> x.x >> x.y;
        cout << (locate(s, x) >= 0 ? "YES\n" : "NO\n");
    }
}

```

```
}  
}
```

4.5 Python

```
import sys  
  
def cross(a, b, c=None):  
    if c is None:  
        return a[0] * b[1] - a[1] * b[0]  
    return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])  
  
def hull(points):  
    points = sorted(set(points))  
    if len(points) <= 1:  
        return points  
    lower = []  
    for point in points:  
        while len(lower) >= 2 and cross(lower[-2], lower[-1], point) <= 0:  
            lower.pop()  
        lower.append(point)  
    upper = []  
    for point in reversed(points):  
        while len(upper) >= 2 and cross(upper[-2], upper[-1], point) <= 0:  
            upper.pop()  
        upper.append(point)  
    return lower[:-1] + upper[:-1]  
  
def normalize(p):  
    if len(p) > 2 and cross(p[0], p[1], p[2]) < 0:  
        p.reverse()  
    at = min(range(len(p)), key=lambda i: (p[i][1], p[i][0]))  
    return p[at:] + p[:at]  
  
def minkowski(a, b):  
    a = normalize(a)  
    b = normalize(b)  
    if len(a) < 3 or len(b) < 3:  
        return hull([(x[0] + y[0], x[1] + y[1]) for x in a for y in b])  
    n, m = len(a), len(b)  
    i = j = 0  
    r = [(a[0][0] + b[0][0], a[0][1] + b[0][1])]  # start with (0,0)  
    while i < n or j < m:  
        ea = (  

```

```

        (a[(i + 1) % n][0] - a[i][0], a[(i + 1) % n][1] - a[i][1])
        if i < n
        else (0, 0)
    )
    eb = (
        (b[(j + 1) % m][0] - b[j][0], b[(j + 1) % m][1] - b[j][1])
        if j < m
        else (0, 0)
    )
    z = cross(ea, eb) if i < n and j < m else 0
    if j == m or i < n and z > 0:
        step = ea
        i += 1
    elif i == n or z < 0:
        step = eb
        j += 1
    else:
        step = (ea[0] + eb[0], ea[1] + eb[1])
        i += 1
        j += 1
    r.append((r[-1][0] + step[0], r[-1][1] + step[1]))
return hull(r[:-1])

```

```

def locate(p, q):
    n = len(p)
    if n == 1:
        return 0 if p[0] == q else -1
    if n == 2:
        return (
            0
            if cross(p[0], p[1], q) == 0
            and min(p[0][0], p[1][0]) <= q[0] <= max(p[0][0], p[1][0])
            and min(p[0][1], p[1][1]) <= q[1] <= max(p[0][1], p[1][1])
            else -1
        )
    a, b = cross(p[0], p[1], q), cross(p[0], p[-1], q)
    if a < 0 or b > 0:
        return -1
    if a == 0:
        return (
            0
            if (q[0] - p[0][0]) ** 2 + (q[1] - p[0][1]) ** 2
            <= (p[1][0] - p[0][0]) ** 2 + (p[1][1] - p[0][1]) ** 2
            else -1
        )
    if b == 0:
        return (
            0
            if (q[0] - p[0][0]) ** 2 + (q[1] - p[0][1]) ** 2
            <= (p[-1][0] - p[0][0]) ** 2 + (p[-1][1] - p[0][1]) ** 2

```

```

        else -1
    )
    l, r = 1, n - 1
    while r - l > 1:
        mid = (l + r) // 2
        if cross(p[0], p[mid], q) >= 0:
            l = mid
        else:
            r = mid
    z = cross(p[l], p[(l + 1) % n], q)
    return -1 if z < 0 else (0 if z == 0 else 1)

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    it = iter(data)
    n, m, q = next(it), next(it), next(it)
    a = [(next(it), next(it)) for _ in range(n)]
    b = [(next(it), next(it)) for _ in range(m)]
    s = minkowski(a, b)
    print(
        "\n".join(
            "YES" if locate(s, (next(it), next(it))) >= 0 else "NO"
            for _ in range(q)
        )
    )

if __name__ == "__main__":
    main()

```

5 D. Convex Polygon Distance

5.1 Restatement and decisive reduction

The distance is the minimum norm of $\mathbf{a}-\mathbf{b}$ over $\mathbf{a}$ in A and $\mathbf{b}$ in B . The set of all such differences is the convex polygon

$$A - B = A + (-B).$$

Thus the problem becomes: construct one Minkowski sum and find the distance from the origin to it. If the origin is inside or on its boundary, the original polygons intersect or touch and the answer is zero.

5.2 Algorithm

Rotate A and $-B$ to their lowest-leftmost vertices. Their CCW edge vectors are cyclically sorted by angle. Merge the two lists, adding equal-direction vectors, to construct $A-B$ in linear time.

Test the origin against every directed boundary edge. For a CCW polygon it is inside exactly when it never lies strictly to the right. Otherwise, project the origin onto every boundary segment, clamp the projection parameter to $[0, 1]$, and take the minimum Euclidean distance.

5.3 Correctness proof

Lemma 1. The merged polygon is exactly $A-B$.

Proof. Negation is a 180-degree rotation, so $-B$ remains convex and CCW. Support functions add under Minkowski sums. Traversing the two angle-sorted edge lists in merged order traces every support-direction change of $A+(-B)$ exactly once, beginning at the sum of their lowest vertices. QED.

Lemma 2. A and B intersect exactly when the origin belongs to $A-B$.

Proof. The origin belongs to $A-B$ exactly when $\mathbf{a}-\mathbf{b}=\mathbf{0}$ for some points, equivalently when the same point belongs to both polygons. QED.

Lemma 3. If the origin is outside $A-B$, the minimum checked segment distance is its distance to the polygon.

Proof. A closest point of a closed convex polygon lies on its boundary. The boundary is the union of the checked segments, and orthogonal projection clamped to one segment returns the closest point on that segment. Taking the minimum therefore returns the closest boundary point. QED.

Together, the lemmas prove that the algorithm returns the minimum distance between the original polygons.

5.4 Complexity, mistakes, and tests

The construction and boundary scan take $O(n+m)$ time and memory. Integer orientation uses wide exact arithmetic; only the final projection uses floating point.

Common mistakes are reversing B's vertex order unnecessarily after negation, failing to merge parallel edge vectors, treating a boundary origin as outside, and measuring only vertex-to-vertex distance. Test overlapping polygons, touching edges, touching vertices, parallel closest edges, and a closest vertex-to-edge projection.

5.5 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
using int128 = __int128_t;

struct Point {
    int64 x, y;
    Point operator+(Point other) const { return {x + other.x, y + other.y}; }
    Point operator-(Point other) const { return {x - other.x, y - other.y}; }
};

int128 cross(Point a, Point b) {
    return int128(a.x) * b.y - int128(a.y) * b.x;
}

vector<Point> normalize(vector<Point> polygon) {
    int start = min_element(polygon.begin(), polygon.end(), [](Point a, Point b) {
        return pair(a.y, a.x) < pair(b.y, b.x);
    }) - polygon.begin();
    rotate(polygon.begin(), polygon.begin() + start, polygon.end());
    return polygon;
}

vector<Point> minkowski(vector<Point> a, vector<Point> b) {
    a = normalize(move(a));
    b = normalize(move(b));
    int n = a.size(), m = b.size();
    vector<Point> edge_a(n), edge_b(m);
    for (int i = 0; i < n; ++i) edge_a[i] = a[(i + 1) % n] - a[i];
    for (int i = 0; i < m; ++i) edge_b[i] = b[(i + 1) % m] - b[i];

    vector<Point> result = {a[0] + b[0]};
    int i = 0, j = 0;
    while (i < n || j < m) {
        Point step;
        if (j == m || (i < n && cross(edge_a[i], edge_b[j]) > 0)) {
            step = edge_a[i++];
        }
    }
}
```

```

    } else if (i == n || cross(edge_a[i], edge_b[j]) < 0) {
        step = edge_b[j++];
    } else {
        step = edge_a[i++] + edge_b[j++];
    }
    result.push_back(result.back() + step);
}
result.pop_back();
return result;
}

long double segment_distance(Point a, Point b) {
    long double dx = b.x - a.x;
    long double dy = b.y - a.y;
    long double denominator = dx * dx + dy * dy;
    long double parameter = -(a.x * dx + a.y * dy) / denominator;
    parameter = max((long double)0, min((long double)1, parameter));
    long double x = a.x + parameter * dx;
    long double y = a.y + parameter * dy;
    return sqrt(x * x + y * y);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n;
    vector<Point> a(n);
    for (auto& point : a) cin >> point.x >> point.y;
    cin >> m;
    vector<Point> negative_b(m);
    for (auto& point : negative_b) {
        cin >> point.x >> point.y;
        point.x = -point.x;
        point.y = -point.y;
    }

    vector<Point> difference = minkowski(move(a), move(negative_b));
    bool contains_origin = true;
    for (int i = 0; i < (int)difference.size(); ++i) {
        Point edge = difference[(i + 1) % difference.size()] - difference[i];
        Point toward_origin{-difference[i].x, -difference[i].y};
        if (cross(edge, toward_origin) < 0) contains_origin = false;
    }

    long double answer = 0;
    if (!contains_origin) {
        answer = numeric_limits<long double>::infinity();
        for (int i = 0; i < (int)difference.size(); ++i) {
            answer = min(answer, segment_distance(

```

```

        difference[i], difference[(i + 1) % difference.size()]);
    }
}
cout << fixed << setprecision(12) << answer << '\n';
}

```

5.6 Python

```

import math
import sys

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def subtract(a, b):
    return a[0] - b[0], a[1] - b[1]

def normalize(polygon):
    start = min(range(len(polygon)), key=lambda i: (polygon[i][1], polygon[i][0]))
    return polygon[start:] + polygon[:start]

def minkowski(a, b):
    a = normalize(a)
    b = normalize(b)
    edge_a = [subtract(a[(i + 1) % len(a)], a[i]) for i in range(len(a))]
    edge_b = [subtract(b[(i + 1) % len(b)], b[i]) for i in range(len(b))]
    result = [(a[0][0] + b[0][0], a[0][1] + b[0][1])]
    i = j = 0
    while i < len(edge_a) or j < len(edge_b):
        if j == len(edge_b) or (
            i < len(edge_a) and cross(edge_a[i], edge_b[j]) > 0
        ):
            step = edge_a[i]
            i += 1
        elif i == len(edge_a) or cross(edge_a[i], edge_b[j]) < 0:
            step = edge_b[j]
            j += 1
        else:
            step = (edge_a[i][0] + edge_b[j][0], edge_a[i][1] + edge_b[j][1])
            i += 1
            j += 1
        result.append((result[-1][0] + step[0], result[-1][1] + step[1]))
    result.pop()
    return result

```

```

def segment_distance(a, b):
    dx, dy = b[0] - a[0], b[1] - a[1]
    parameter = -(a[0] * dx + a[1] * dy) / (dx * dx + dy * dy)
    parameter = min(1.0, max(0.0, parameter))
    x = a[0] + parameter * dx
    y = a[1] + parameter * dy
    return math.hypot(x, y)

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    at = 0
    n = data[at]
    at += 1
    a = [(data[at + 2 * i], data[at + 2 * i + 1]) for i in range(n)]
    at += 2 * n
    m = data[at]
    at += 1
    negative_b = [(-data[at + 2 * i], -data[at + 2 * i + 1]) for i in range(m)]
    difference = minkowski(a, negative_b)
    contains_origin = all(
        cross(
            subtract(difference[(i + 1) % len(difference)], difference[i]),
            (-difference[i][0], -difference[i][1]),
        ) >= 0
        for i in range(len(difference))
    )
    answer = 0.0 if contains_origin else min(
        segment_distance(difference[i], difference[(i + 1) % len(difference)])
        for i in range(len(difference))
    )
    print(f"{answer:.12f}")

if __name__ == "__main__":
    main()

```

6 Stress testing and carry-forward

The D oracle does not build a Minkowski polygon. On tiny polygons it checks intersection directly and compares every vertex with every opposite edge. The reusable discovery move is algebraic: turn a relation between two convex sets into a query about the origin in their sum or difference.